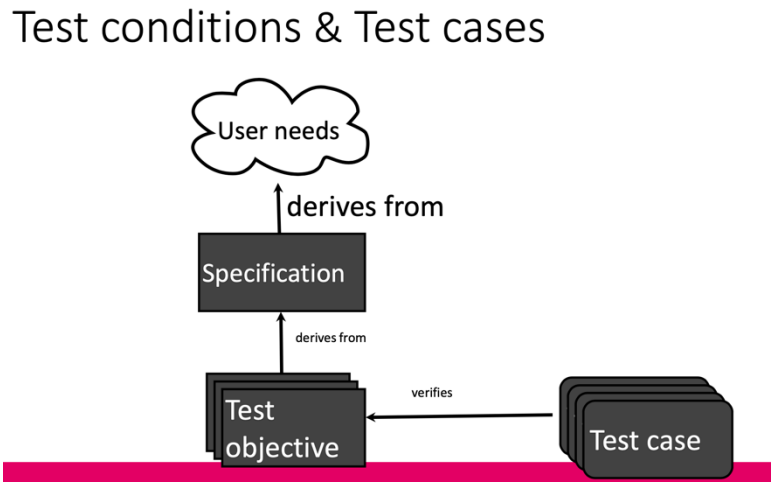
Comment est ce qu'on va aller vérifier qu'on a bien réalisé le bon logiciel et qu'on a réalisé le logiciel de la bonne façon ?



le plus difficile est de capturer le User requirement. À partir de là, on va demander aux business analyste d'étudier le User requirement (needs) et de discuter avec le cliente et d'en créer des spécifications. Le spécification est un View que nous avons des besoins du client qui peut ou ne peut être le bon (notre propre interprétation de la réalité) et à partir de là on va encore une fois en dériver des objective de test et après à partir de là on va écrire des cas de Test qui vont vérifier ces objective de test. Ce qui import c'est que les spécifications sont déjà une perte de l'information entre ce que l'utilisateur a dans ça tête et les spécification écrit par une business analyste Et test objective en fais une autre.

Qui a créé le Test case ? c'est fait par les managers. La personne le moins bien placé pour faire les 4 test est le développeur. Parce que si c'est la même personne qui écrit les 4 tests et écrit le programme aura le même billet qui va être appliquer aux deux.

Vocabulaire :

Test objective: ce qu'il faut tester (soit qqch de fonctionnalité, NFR)
Test Case: vérifie un objectif de test et pour cela on va créer un ensemble de données d'entrée. On met le système dans un état particulier que on appelle pré-conditions, après on exécute ce qu'on veut vérifier, et on va vérifier les résultat en sorti et les post-conditions. (on va vérifier les post-conditions aussi pour s'assurer qu'on n'a pas d'effet de bord)
Procédure de test: procédure de configuration, d'exécution et d'évaluation du résultat d'une séquence de cas de test.
Plan de test: description de scope, de l'approche, des ressources et l'organisation en terme temporelle des activités de test. C'est vraiment un travail de management qui est très important.

Que ce que c'est faire de testing ?

Les tests comprennent les activitées suivantes:

1. **Planification et contrôle des tests** (organiser et manager des tests)
   - Définition ce qu'on veut tester
   - Avoir une planification de haut niveau
   - Avoir des méthodes pour contrôler et faire un report des bugs

2. **Analyser et faire de désigne de mon test** (il faut avoir des données prêtes pour le test) :
   On va se placer dans le cas de test de haut niveau et on met en place une architecture de test.
   Exemple : quand je suis en train de faire de test de performance, je vais probablement avoir une architecture de test qui sera différent de test fonctionnel parce que dans le cas de test performance, je rajoute une machine qui est bcq plus performante et peut-être, j'aurais des données de teste qui sont bcq plus volumineuses que dans le cas de test fonctionnel. À partir de là, je peux définir mes cas de tests de haut niveau.

3. **Implémenter des tests**
   - mettre des priorité (la plus part de temps on arrive pas à tout tester donc il faut définir des priorités).
   - Créer une procédure de test.
   - Création de test harness (environnement de test) pour que mes testeur soient dans l'état le plus efficace pour créer les 4 tests et les exécuter.
   - Contrôle de la traçabilité bidirectionnelle càd on a les 4 tests, on sais sur quel environnement ils ont été exécutés, on connais le résultat de l'exécution alors on a une traçabilité de notre test qui nous permet de remonter de l'erreur jusqu'au tout en haut.
   - Exécution de la procédure du test
   - On doit collecter les résultat
   - Comparer les résultats effectives et attendus
   - S'il a une erreur, il faut tracer la raison
   - Répétez cette séquence aussi long temps qu'il rest des défauts à corriger.

4. **il faut Évaluer des critères de sortie et les rapports**
   pour ça il y a plein de méthode différentes et le meilleur est de bon sens.
   - Collecter les données produites par les exécutions de tests et les présenter aux parties prenantes afin qu'elles puissent prendre une décision de continuer ou pas. Quel est le niveau de couverture de mon code ? même si c'est à 90% ça ne veut pas dire que je peut m'arrêter là. J'ai peut-être créer énorméments de bugs.
   - ensuit on Détermine les zones les plus exposé par les bugs. On regarde dans les mois passé tous les bugs qui nous ont échappés (ce que le cliente a trouver) et on regarde sur quel module il s'applique(peut être le composante est très complexe à tester ou peut-être un problème de l'équipe parce que c'est toujours la même équipe).

   Il faut regarder cas par cas pour savoir comment on évalue la confiance que l'on a dans le logiciel et par rapport cette confiance, on peut dire au client si on continue ou on arrête de tester.

5. **Tester les activités de clôture :**
   Une fois qu'on a déterminé qu'on s'arrête il faut s'occuper des activités de clôture càd
   - comment est-ce qu'on va améliorer l'infrastructure de test par rapport à cette expérience et comment est-ce qu'on va créer guideline par rapport à cette expérience.
   - On va nettoyer l'infrastructure de test pour la prochaine exécution.

Combien de tests suffisent-ils?

- **Des risques**
- on ne test pas humain life et le milliard de dollar de la même façon. Il est **important de savoir dans quel cas il faut mettre plus de test**. Dans le cas des humain la plupart de temps on a deux équipes différentes de teste. Tous ça c'est très cher. C'est pour ça, qu'il faut décider à mieux placé les ressources.
- Nous, en tant que développeur, en tant que testeur, ce n'est pas notre rôle de décider qu'on doit arrêter notre job est **de ramener suffisamment d'informations et de les afficher de façon suffisamment clair** pour dirigeantes pour qu'elles puissent décider de continuer ou d'arrêter.

7 principes dans les tests :

1) Les tests prouvent la présence de bugs mais Les tests NE PROUVENT PAS l'absence de bugs (peut-être nos tests sont nuls).

2) **Exhaustive testing** (tester tout les combinaisons possibles pour s'assure de l'absence de test est impossible).
   Exemple: pour un petit programme comme :
   Long I, Long j , Long[16]k on a $2^{64} * 2^{64} *(2^{64})^{16} = ??$ de cas de test à écrire.
   Alors on peut essayer de restreindre nos cas de test par exemple 3*3*5*3, c'est quelque chose de faisable mais quand on écrit 3*3*5*3, il faut qu'on sache que à quel point c'est représentative par rapport à l'ensembles des cas de test.

3) Plus tôt les tests sont fait, mieux c'est ---> moins de temps perdu

4) Recherchez la densité des bugs. Les tests doivent se concentrer sur les modules avec une densité de bugs plus important.

5) Les tests vieillissent mal. En d'autres termes, il faut souvent réécrire les tests, souvent le revoir

6) Les tests ne sont pas effectués de la même manière dans les applications critiques pour la sécurité et dans les sites Web de commerce électronique. N'oubliez pas d'évaluer les risques et les enjeux.

7) Aucun défaut ne signifie pas que le système est prêt. Cela signifie qu'il répond a la spécification définie par notre test mais pas nécessairement aux requirments(besoins) de l'utilisateur.

Comment on fais les tests ?

(On va les expliquer par la suit)

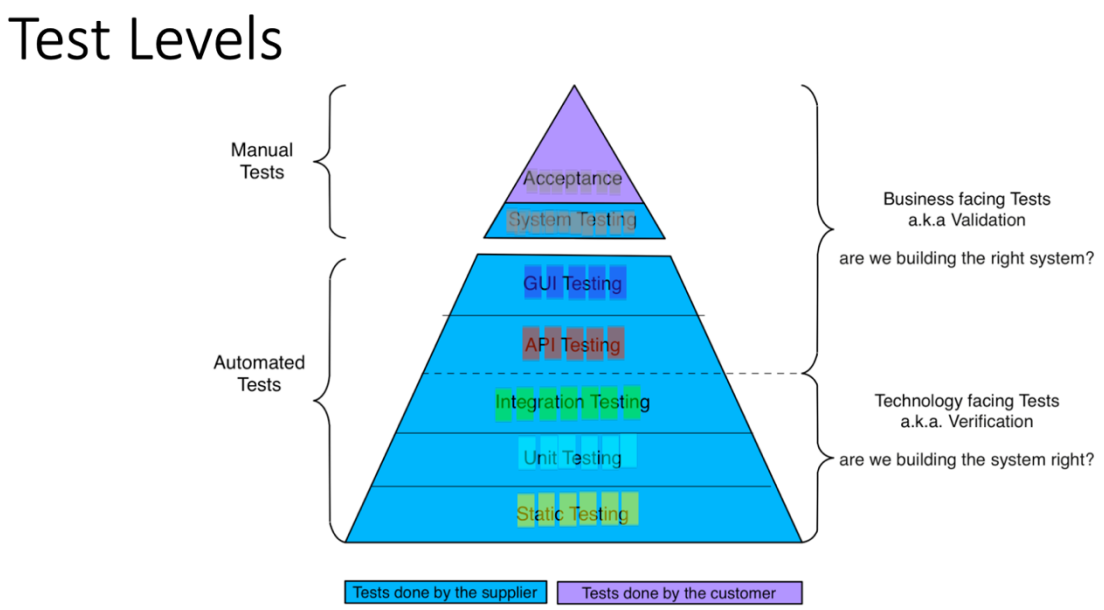
Test Levels

Components Testing

Integration Testing

System Testing

Acceptance Testing



static Testing: Pour les test les plus bas, on passe plus de temps. Plus on monte dans la pyramide, moins on passe du temps
Au niveau de static Testing on trouve pas des erreurs fonctionnel. On trouve plus tôt les erreurs courantes. Par exemple, des boucles un peu bizarre ou par exemple des choses comme a = b à la place de a==b. Ce test ne coûte rien car c'est un automate qui le fait.

Unit Testing: on va tester des classes ,des packages,

Integration Testing : Ici on va mettre des composantes. Jusqu'à ce niveau là, sont des types technologie donc c'est de la vérification. Est-ce qu'on a fait ce système correctement ?

API Testing(Application programming interface): on test en se mettant en dehors du produit. on est plus sur les test en boîte noir, on voit pas l'intérieur du produit.

GUI Testing: on test en se mettant en dehors du produit ici aussi.

Remarque: Jusqu'à là, tous les tests sont automatisés.

System Testing: c'est une test manuel où on test l'entièreté du système d'un point de vue technique.

User acceptance Testing: ce test est fait par le client.

Compenant Testing(Unit Testing ou Module Testing): on test là des composantes de bas niveau. Comme j'ai expliqué précédemment, on va tester des classe, des packages pas plus. Ici on test un composant à la fois, c'est pour ça qu'on l'appelle unit testing. Ça utilise des mocks, des stubs, des fakes. Cela veut dire, on remplace certains autres composantes dont on dépende par des fausse classes ou des faux objets qui vont simuler ces autres composantes comme ça on sait s'il y a un bug. Ce n'est pas liée aux autres composantes et bien aux composantes que je suis en train de tester. On peut faire des tests fonctionnels et non fonctionnel qui sont fait par les programmeurs.

Integration Testing: intégration testing ça se fait après component testing. ici on peut tester un peux plus d'architecteur, un peux plus de use case et on va prend plusieurs composantes(sous système) ensemble et on va les tester ensemble. C'est important quand-t-on fait du test d'intégration, certains des composantes qu'on va les mettre ensemble peut encore avoir des faux composantes parce qu'il sont pas encore prêt.
Mettre trop des composantes ensemble rend la traçabilité plus compliqué c.à.d, ça devient plus compliqué de savoir d'où vient l'erreur. La plus part du temps, les tests d'intégration sont fait par les développeurs ou par l'intégration manager.

Remarque : On saute le cas API parce que c'est assez similaire à intégration test mais en boîte noir

System Testing: on prend la totalité , on test la système dans sa totalité d'un point de vu technique. Ca se fait après l'intégration testing. Là, il y'a plus de mocks et l'environnement de test doit être aussi proche que possible de la production et souvent fait par une équipe différent pour éviter des billets et aussi pour traiter de tout la partie infrastructure .

User acceptance Testing: Là c'est fonctionnel , on se pose la question comment va se passer pour le client. Là on va tout tester. Ce se fait après le système testing. L'idée est de donner suffisamment d'informations à l'utilisateur pour qu'il puisse prendre de décision de go/no go pour la production. C'est lui (l'utilisateur) qui valide qu'on peux aller pour la production et aucun d'autre.

Une fois que fait tous ça il va falloir de catégoriser tous nos tests.

Test types :

1. **Functional**
2. **Structure / Architecture** --> est-ce que je dois m'intégrer avec d'autre composante ?
3. **Non-Functional** -- > est ce que la performance est au rendez-vous ? par exmple si je multiplie par 2 mon nombre d'utilisateur ma Platform va tenir le Coup ?
4. **Change-related** -- > on écrit le test spécifiquement pour un changement qu'on vient de l'effectuer . c'est là qu'il le plus grand risque d'introduire des nouveaux bugs parce que on a crée un changement.

Remarque : On ne peut pas dire que la partie fonctionnel est plus important que la partie Non fonctionnel ou à l'inverse cela vraiment dépond du cas . par exemple dan le cas de **UBER,** la partie fonctionnel est relativement simple (est ce qu'il y'a un conducteur dans ma zone? Oui ou No . est qu'il veux prendre ma cours ? oui ou No. Après je paie et on se voit plus). Mais pour envoyer une sond spécial dans l'space le calcule à faire(partie fonctionnel) est énormément plus complexe à implémenter .

Technical tests

1. **boîte blanche :** on prend le logiciel comme une boîte et on peux voir l'intérieur càd on peut comprendre comme c'est fais à l'intérieur. On utilise notre connaissance du logiciel pour écrire la test.

2. **boîte noir** : ici la seule chose qu'on connais c'est le contrait avec notre composante tous le reste on les ignore. Là c'est également à l'intérieur. on se comporte comme cliente de base.

remarque: On ne peut pas dire que le test blanche est plus important que le test noir ou à l'inverse cela vraiment dépond du cas. Une bon politique de test a à la fois du test en boîte blanche et du test en boîte noir.

Une fois qu'on sait qu'on est en boîte blanche ou en boîte noir, on va pouvoir écrire différent Technique de test

Technical tests :

1. **Static : on n'a pas besoin de exécuter le code pour le tester**
   - Review / Audit -- > revoir le code on regarde le code

   - Static analysis -- > il s'agit d'inspecter le code, des exécution qu'on appelle symbolique et regarder les erreurs que on rencontre. Même sans jamais exécuter le code certain outil qu'on appelle des outils d'analyse des codes static Peuvent détecter les bugs potentiels. C'est gratuit ce test.

2. **Dynamic** : **on a besoin d'exécuter le code pour le tester**
   - White box -- > je connais la structure et j'écris de code qui sont en fonction de l'structure du logiciel

   - Black box -- > je ne connais pas la structure, j'écrit de test comme une cliente qui n'a aucun connaissance interne du logiciel. Ce test est bcq plus compliqué à écrire car souvent il faut mettre bcq de structure et infrastructure en place pour faire le test. On peut pas tester tous les valeurs possibles. du coup on partion l'space de test et on fait des classes équivalentes (par exemple 3 class randomes -1 ,0 et 1). Si ça fonction pas il faut repasser à test blanche pour voir ce qui se passe Après on pass à Boundary test(test au bornes) pour voir ce voir ce qui se passe quand logiciel atteindre … critique. Par exemple le system tombe sèche, est qu'il peut redémarrer ?
     et en fin quelque chose de très pratique c'est de créé de décision table. Décision table c'est un ensemble de contraintes sur une ligne et appliquer ces contraintes une par une pour voir ce qu'on a en sortie c'est bien ce qu'on souhaite.

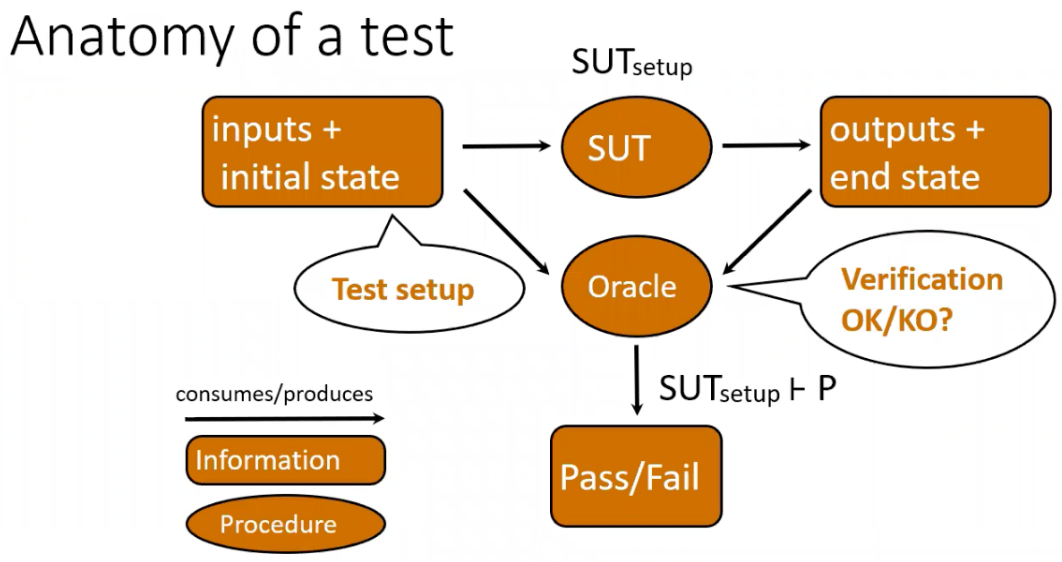
   - Code coverage -- > calculer le pourcentage de code qui a été testé par les tests sur la Structeure. Permet d'écrire de test en visant la partie qui sont mal couverte. Comme ils sont mal couvert c'est là qu'on risque d'avoir surprise. Il y'a un grosse différente entre passer sur tous les statements et faire tous les tests de tous les statements. Le fait d'avoir une couverture de 80% sur les statements c'est pas forcement intéressant c'est bcq mieux si on a 80% sur les conditions également parce que on peut tester tous les lignes mais pas tous les conditions(au niveau de qualité).

Dynamic Testing : c'est en boite blanche donc en peut s'occuper également de coverage d'un point de vu de décision et de statement.

Que ce que ça veut dire d'écrire un test ?

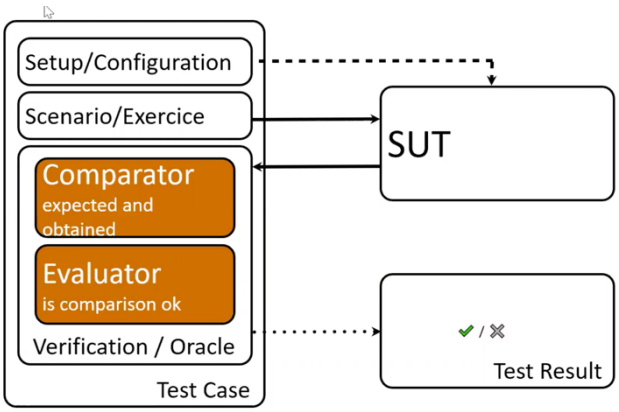
La première chose qu'il faut garder en tête quand on écrit de test c'est de quoi il est composé ? C'est quoi les différente patrie que compose un test ?

Pour tester il faut un ensemble de inputs, un ensemble de outputs et un oracle pour savoir cet état final est bien l'état qui été attendu.



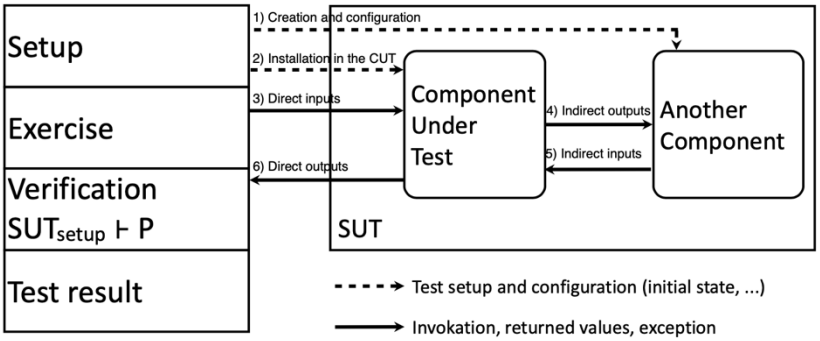
Test Case :

1. Créer une configuration /setup
2. Le scenario càd les différentes étapes que l'on veut tester
3. Ajouter un **comparateur** qui permet de comparer l'état attendu avec l'état obtenue
4. Ajouter un **Évaluateur** qui va nous dire, est-ce que la différence entre l'état attendu et l'état obtenu est une bonne chose (parce que si on a une exception dans notre code, cela ne veut pas dire qu'on a un problème)
5. Ce setup va s'appliquer à software and test (SUT), puis on va jouer les différents événements (le met dans certain nombre d'état et faire changer d'état). Ce résultat va arriver à comparateur et à l'aide de l'évaluateur il va déterminer si ce test est un succès ou pas.

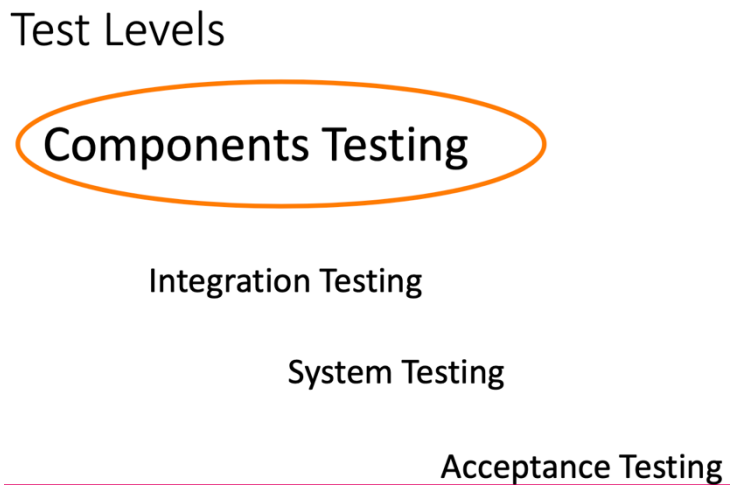


Important : un même test peut être vu en tant qu'un succès par un oracle et en tant qu'un échec par un autre oracle.

Important : On ne test pas un seul composant. La plupart du temps, on test plusieurs composants ensemble -- > il faut créer l'autre composant, l'injecter dans notre composante puis lancer le test.



De quoi aurions-nous besoin pour écrire différentes tests?



Test components (niveau unit test) : On veut pouvoir garantir lorsqu'on test qqch. On va avoir l'impact liée à ce qu'on est en train de tester mais pas quelque chose d'autre qui vient perturber notre test. C'est pour ça que pour tester en isolation, nous utilisons le technique Mock ou fackes qu'on appelle test doubles. On utilise test doubles pour améliorer l'isolation du test. **Par tout, on veut garantir que ça n'aura pas d'effet de bure au lieu d'utiliser un autre composant. On utilise un test double et celui-là on maîtrise complétement son comportement donc ne risque pas d'influencer sur notre test sinon on sera plus au niveau de composant.**

Types de Test doubles ;

1. **Dummy object :** n'a aucun comportement. Il est là pour faire en sort que ça compile.
2. **Fake : je peux tester que l'essentiel de ma logique(test) fonctionne et que je n'ai pas de problème major.** Ce n'est pas autant que l'objet complet mais a certain comportants, ont créé un fake et on l'injecte à notre composant under test, le reste va suivre tout à fait normalement.
3. **Test Stub :** test stub va nous envoyer au cas limite càd on va plutôt tester les cas limites, le reste est exactement identique

4. **Test Mock :** vérification sur l'état de Mock càd le nombre de foi que le méthode inc a été appelé. L'intérêt est de pouvoir s'assurer les communications (en boîte blanche, comment se passe le test)
5. **Test spy :** pour un vrai logiciel, on a une 100èn appelle donc Test Mock c'est trop fatiguant du coup on a inventé le **Test spy**. On utilise un proxy et on enregistre tous les indirects inputs et outputs. On peut prendre un composant existant, on met un test spy au milieu et on peut enregistrer tous ce qui se passe lors de comportement normal d'une application.

Test de régression : j'ai une nouvelle version de mon logiciel, je refais le même test que le mois d'avant, ce que marchais avant il y'a un mois marche toujours → je n'ai rien cassé au passage.